

Glosario - Semana 08

Arquitectura en Capas Completa

A

Application Layer (Capa de Aplicación) Capa intermedia que contiene la lógica de negocio. Los Services orquestan operaciones entre múltiples repositories y aplican reglas de negocio.

```
class OrderService:
    def __init__(self, order_repo, user_repo, product_repo):
        self.order_repo = order_repo
        self.user_repo = user_repo
        self.product_repo = product_repo
```

B

BaseRepository Clase genérica que implementa operaciones CRUD comunes. Otros repositories heredan de ella para evitar duplicación de código.

```
class BaseRepository(Generic[T]):
    def get_by_id(self, entity_id: int) -> T | None:
        return self.db.get(self.model, entity_id)
```

Business Logic (Lógica de Negocio) Reglas y procesos específicos del dominio. Ejemplo: calcular impuestos, validar stock, aplicar descuentos. Vive en la capa de servicios.

C

Capa (Layer) División horizontal de responsabilidades en una aplicación. Cada capa tiene un propósito específico y se comunica solo con capas adyacentes.

ConfigDict Configuración de modelos Pydantic v2 que reemplaza la clase `Config` interna.

```
class UserResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)
```

Conflict Error Error HTTP 409 que indica que la operación no puede completarse porque entra en conflicto con el estado actual del recurso (ej: email duplicado).

D

Data Access Layer (Capa de Acceso a Datos) Capa responsable de la persistencia. Los Repositories encapsulan las operaciones con la base de datos.

Data Transfer Object (DTO) Objeto que transporta datos entre capas. En FastAPI, los schemas Pydantic actúan como DTOs.

```
class UserCreate(BaseModel): # DTO de entrada
    email: EmailStr
    password: str

class UserResponse(BaseModel): # DTO de salida
```

```
id: int
email: str
```

Dependency Injection (DI) Patrón donde las dependencias se proporcionan desde afuera en lugar de crearse internamente. FastAPI lo implementa con `Depends()`.

```
def get_user_service(
    repo: UserRepository = Depends(get_user_repository)
) -> UserService:
    return UserService(repo)
```

E

Entity (Entidad) Objeto del dominio con identidad única, típicamente mapeado a una tabla de base de datos. En SQLAlchemy son los modelos ORM.

Exception Handler Función que intercepta excepciones y las convierte en respuestas HTTP apropiadas.

```
@app.exception_handler(AppException)
async def app_exception_handler(request, exc):
    return JSONResponse(status_code=exc.status_code, ...)
```

exclude_unset Opción de Pydantic que excluye campos no proporcionados, útil para actualizaciones parciales (PATCH).

```
update_data = dto.model_dump(exclude_unset=True)
```

F

Factory Function Función que crea y retorna instancias de objetos. En FastAPI se usan como dependencias para crear services y repositories.

```
def get_user_repository(db: Session = Depends(get_db)) -> UserRepository:
    return UserRepository(db)
```

from_attributes Configuración de Pydantic que permite crear modelos desde objetos ORM (antes `orm_mode = True`).

G

Generic Repository Repository que usa genéricos de Python para ser reutilizable con diferentes tipos de entidades.

```
T = TypeVar("T")

class BaseRepository(Generic[T]):
    def __init__(self, db: Session, model: Type[T]):
        self.model = model
```

H

Handler Función que maneja un evento o solicitud específica. En FastAPI, los exception handlers manejan errores globalmente.

I

IntegrityError Excepción de SQLAlchemy cuando se viola una restricción de base de datos (unique, foreign key, etc.).

```
except IntegrityError:
    self.db.rollback()
    raise UserAlreadyExistsError(email)
```

J

joinedload Estrategia de SQLAlchemy para cargar relaciones en una sola consulta SQL (eager loading).

```
stmt = select(Order).options(joinedload(Order.items))
```

L

Layered Architecture (Arquitectura en Capas) Patrón arquitectónico que organiza el código en capas horizontales con responsabilidades específicas: Presentación → Aplicación → Datos.

M

Mapper Clase que convierte entre diferentes representaciones de datos (Entity ↔ DTO).

```
class UserMapper:
    @staticmethod
    def to_entity(dto: UserCreate, password_hash: str) -> User:
        return User(email=dto.email, password_hash=password_hash, ...)

    @staticmethod
    def to_response(entity: User) -> UserResponse:
        return UserResponse.model_validate(entity)
```

model_validate Método de Pydantic v2 para crear una instancia desde un objeto (reemplaza `from_orm()`).

```
UserResponse.model_validate(user_entity)
```

O

Orchestration (Orquestación) Coordinación de múltiples operaciones o servicios. Los Services orquestan repositories.

```
class OrderService:
    def create(self, data):
        user = self.user_repo.get_by_id(data.user_id) # Validar
        # ... validar productos, calcular totales, reducir stock
        return self.order_repo.add(order) # Guardar
```

P

Presentation Layer (Capa de Presentación) Capa que maneja la interacción con el exterior (HTTP). Los Routers de FastAPI pertenecen a esta capa.

R

Repository Pattern Patrón que abstrae la lógica de acceso a datos, proporcionando una interfaz de colección para las entidades.

```
class UserRepository:
    def get_by_id(self, user_id: int) -> User | None: ...
    def get_by_email(self, email: str) -> User | None: ...
    def add(self, user: User) -> User: ...
```

Response Model Schema Pydantic que define la estructura de la respuesta HTTP en FastAPI.

```
@router.get("/{user_id}", response_model=UserResponse)
def get_user(user_id: int): ...
```

S

Separation of Concerns (Separación de Responsabilidades) Principio que establece que cada módulo debe tener una única responsabilidad. Base de la arquitectura en capas.

Service Layer Capa que contiene la lógica de negocio y orquesta operaciones entre repositories.

Snapshot Copia de datos en un momento específico. En pedidos, se guarda el nombre y precio del producto al momento de la compra.

```
class OrderItem(Base):
    product_name: Mapped[str] # Snapshot, no relación
    unit_price: Mapped[float] # Precio al momento de compra
```

T

Transaction (Transacción) Conjunto de operaciones que deben completarse todas o ninguna. SQLAlchemy maneja transacciones automáticamente con `commit()` y `rollback()`.

U

Unit of Work Patrón que mantiene una lista de objetos afectados por una transacción. La Session de SQLAlchemy implementa este patrón.

V

Validation Error Error HTTP 422 que indica datos de entrada inválidos. Pydantic los genera automáticamente, pero también se pueden crear personalizados.

```
class InsufficientStockError(ValidationError):
    def __init__(self, product_id, requested, available):
        super().__init__(
            message=f"Insufficient stock...",
            error_code="INSUFFICIENT_STOCK"
        )
```



Resumen de Capas

Capa	Componente	Responsabilidad
Presentation	Router	HTTP, validación entrada
Application	Service	Lógica de negocio

Data Access	Repository	Persistencia
Cross-cutting	Exception Handler	Manejo de errores
Cross-cutting	Mapper	Conversión de datos

Referencias

- [Martin Fowler - Patterns of Enterprise Architecture](#)
- [FastAPI - Bigger Applications](#)
- [Clean Architecture - Robert C. Martin](#)